

Software Requirements Specification

Clarío

A Multi-Agent Customer Support Triage and Response System

Version 1.0

Date: 25/07/2026

Authors: WEERASEKARA R.V. (230694J), WETTASINGHE V.N. (230701G), WICKRAMARATNA S.I.P. (230703N)

Group ID: Group 23 (Project ID: P04)

Mentor: Dr. Aloka Fernando

Teaching Assistant: Dilanka

Revision History

Date	Version	Description	Author
30/07/2026	1.0	Initial condensed SRS release for the Clario multi-agent customer support platform	Group 23

Table of Contents

1. Introduction	5
1.1 Purpose	5
1.2 Scope	5
1.3 Definitions, Acronyms, and Abbreviations	5
1.4 References	6
1.4.1 Standards	6
1.4.2 Project Documentation	6
1.5 Overview	6
2. Overall Description	7
2.1 Product Perspective	7
2.2 Product Functions	7
2.3 User Classes and Characteristics	8
2.4 Operating Environment	8
2.5 Design and Implementation Constraints	9
2.6 Assumptions and Dependencies	9
3. Specific Requirements	9
3.1 Functionality	9
3.1.1 User Authentication and Authorization	9
3.1.2 Ticket Management	10
3.1.3 Ticket Classification	10
3.1.4 PII Detection and Redaction	11
3.1.5 Supervisor Agent Orchestration	11
3.1.6 Specialist AI Agents	12
3.1.7 Retrieval-Augmented Generation (RAG)	12
3.1.8 Response Validation	13
3.1.9 Human Review and Escalation	13
3.1.10 Knowledge Base Management	14
3.1.11 User Management	14
3.1.12 System Administration	14
3.1.13 Logging and Monitoring	15
3.1.14 Reporting and Analytics	15
3.1.15 AI Model Management	15
3.1.16 Notification Services	16
3.1.17 Configuration Management	16
3.1.18 System Health Management	16
3.2 Usability	16
3.3 Reliability	18
3.4 Performance and Security	20
3.4.1 Performance Requirements	20
3.4.2 Security Requirements	21

3.5 Supportability	23
3.6 Design Constraints	25
3.7 On-line User Documentation and Help System Requirements	26
3.8 Purchased Components	27
3.9 Interfaces	27
3.9.1 User Interfaces	27
3.9.2 Hardware Interfaces	27
3.9.3 Software Interfaces	27
3.9.4 Communications Interfaces	28
3.10 Database Requirements	28
3.11 Licensing, Legal, Copyright, and Other Notices	28
3.12 Applicable Standards	28
4. Supporting Information	28
4.1 Architecture References	28
4.2 Academic References	29

Software Requirements Specification

1. Introduction

This Software Requirements Specification (SRS) defines the complete software requirements for Clario, a multi-agent customer support triage and response system. It specifies the functional and non-functional requirements, external interfaces, system constraints, quality attributes, and acceptance criteria required for implementation, testing, deployment, and maintenance of the system. This document has been prepared following IEEE 29148:2018 and IEEE 830 principles and is condensed from the full project SRS to provide a concise, implementation-ready reference for developers, AI engineers, testers, and project supervisors.

1.1 Purpose

The purpose of this SRS is to provide a precise, implementation-ready description of the Clario system and to establish a common understanding among stakeholders of its expected behavior, interfaces, constraints, and quality attributes. It defines the functional behavior of the platform, specifies measurable non-functional requirements (performance, security, reliability, scalability, maintainability, usability, availability), and provides acceptance criteria against which the implementation can be verified and validated.

1.2 Scope

Clario is an AI-powered customer support triage and response platform that automates the initial stages of customer support while keeping complex, high-risk, or low-confidence cases under human oversight. A LangGraph-orchestrated multi-agent architecture coordinates ticket analysis, domain-specific reasoning, Retrieval-Augmented Generation (RAG), response validation, and escalation. The system automatically analyzes incoming tickets, identifies category, urgency, priority and sentiment, routes tickets to specialist AI agents, retrieves relevant organizational knowledge, generates grounded responses, validates them against policy, escalates unsupported or high-risk cases, and maintains an auditable history of AI reasoning. The system targets educational institutions and customer support environments requiring intelligent ticket processing.

In scope for this release:

- AI-assisted ticket triage and multi-agent orchestration
- Retrieval-Augmented Generation (RAG) and knowledge base management
- Human-in-the-loop review workflows
- Authentication, authorization, and administrative configuration
- Logging and monitoring

Out of scope for this release:

- Full CRM platform integration and enterprise-scale distributed deployment
- Autonomous customer communication without human oversight
- Voice-based and advanced multilingual conversational support
- Continuous online model retraining
- Multi-provider enterprise authentication solutions beyond the defined architecture

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
AI	Artificial Intelligence
Agent	An autonomous software component performing specialized tasks within the orchestration workflow
API	Application Programming Interface
ChromaDB	Vector database used for storing document embeddings supporting RAG
Classification Model	Fine-tuned language model predicting ticket category,

	priority, and sentiment
Clario	The multi-agent customer support triage and response platform described in this document
Fine-Tuned LLM	Parameter-efficient adapted language model used for ticket classification
Human Handoff	Workflow transferring ticket responsibility from AI agents to a human operator
JWT	JSON Web Token used for stateless authentication
Knowledge Base (KB)	Collection of approved support documents used by RAG
LangGraph	Framework used to orchestrate multiple AI agents through explicit state-based workflows
LLM	Large Language Model
LoRA / QLoRA	Low-Rank Adaptation / quantized variant, parameter-efficient fine-tuning methods
PostgreSQL	Relational database management system used for application data storage
RAG	Retrieval-Augmented Generation, combining retrieved knowledge with LLM generation
ShadowMap	AES-256 encrypted mapping of PII to surrogates
SurrogateShield	Deterministic PII masking engine using Regex and spaCy NER
Supervisor Agent	Primary orchestration component coordinating the overall workflow
Ticket	A customer support request submitted through the system
Validation Agent	Component verifying response quality and policy compliance before delivery

1.4 References

1.4.1 Standards

- IEEE Std 29148-2018 – Systems and Software Engineering – Life Cycle Processes – Requirements Engineering
- IEEE Std 830 – Software Requirements Specification
- ISO/IEC/IEEE 12207 – Software Life Cycle Processes

1.4.2 Project Documentation

This SRS is derived from the approved Clario project proposal, system architecture document, backend and frontend architecture, implementation roadmap, AI agent architecture, RAG design, and deployment architecture documents developed for the Clario project (CS3501, Group 23).

1.5 Overview

The remainder of this document is organized as follows. Section 2, Overall Description, provides a high-level overview of Clario, including product perspective, user classes, operating environment, assumptions, dependencies, and design constraints. Section 3, Specific Requirements, defines the complete functional and non-functional requirements, interface specifications, database requirements, security requirements, and quality attributes. Section 4, Supporting Information, contains architecture and academic references.

2. Overall Description

Clario is an AI-powered customer support triage and response platform combining multi-agent orchestration, Retrieval-Augmented Generation (RAG), and a fine-tuned language model to automate the initial stages of customer support while keeping high-risk or uncertain cases under human supervision. It aims to improve response quality, reduce manual workload, provide consistent support, and maintain transparency through auditable AI-assisted decision making.

2.1 Product Perspective

Clario is a standalone, modular customer support orchestration system that acts as an intelligent processing layer between end users submitting support requests and human support personnel resolving them. It receives tickets through a web UI or external API, performs automated analysis, routes requests through specialist AI agents, validates generated responses, and either returns a grounded response draft or escalates to a human representative. The architecture follows a service-oriented, containerized design. Major components include:

- React (Next.js) + TypeScript + Tailwind CSS frontend
- Spring Boot API Gateway (Java 17/21)
- Python FastAPI ML Sidecar (LangGraph orchestration)
- Fine-tuned classification model service
- Retrieval-Augmented Generation (RAG) subsystem
- PostgreSQL relational database and Chroma vector database
- Knowledge base management, logging, and monitoring infrastructure

The initial deployment targets an academic customer support environment, with the Rysera STEM Learning Management System as the primary pilot platform; the architecture avoids Rysera-specific business logic so it can be adapted to other support environments.

2.2 Product Functions

At a high level, Clario provides the following major capabilities:

Ticket Submission and Intake

- Accept customer support requests through the web application and REST APIs.
- Validate incoming requests before processing.
- Generate unique ticket identifiers.
- Record ticket metadata and processing history.

Automated Ticket Analysis

- Detect ticket category and predict priority.
- Identify customer sentiment and estimate classification confidence.
- Support single-domain and multi-domain ticket identification.

PII Protection

- Detect sensitive information within ticket content.
- Mask personally identifiable information before external AI processing.
- Preserve redaction metadata for auditing.

Intelligent Ticket Routing

- Route tickets to one or more specialist AI agents.
- Support dynamic routing decisions based on ticket characteristics.
- Support rerouting when validation identifies an incorrect specialist assignment.

Retrieval-Augmented Generation (RAG)

- Retrieve relevant knowledge base documents and rank them by semantic relevance.
- Build contextual prompts using retrieved knowledge.
- Ensure generated responses remain grounded in approved documentation.

Specialist AI Response Generation

- Generate domain-specific response drafts.
- Support multiple specialist agents operating independently.
- Combine outputs when multiple specialists participate in ticket resolution.

Response Validation

- Verify policy compliance and detect unsupported or hallucinated responses.
- Evaluate response quality and trigger bounded reflection where appropriate.
- Escalate unresolved issues.

Human Review and Escalation

- Escalate urgent, unsupported, or low-confidence cases.
- Produce structured handoff packages.
- Preserve reasoning history for human reviewers.

Knowledge Management

- Maintain versioned knowledge base documents and support vector indexing.
- Manage retrieval metadata and track knowledge source ownership.

Administration

- Manage authentication and authorization; configure system settings.
- Monitor application health; view logs and operational metrics.

2.3 User Classes and Characteristics

User Class	Characteristics	Primary Permissions
Customer / End User	Submits tickets via web app; minimal technical knowledge.	Submit tickets; view status; receive responses.
Customer Support Agent	Reviews escalated tickets; moderate technical knowledge; knows support policy.	Review/modify AI drafts; approve/reject; close tickets.
System Administrator	High technical expertise; manages infrastructure.	Configure settings; manage users; monitor health; manage deployments.
Knowledge Base Administrator	Maintains organizational documentation.	Publish/update articles; manage versions; trigger indexing.
AI Operations Engineer	Responsible for AI lifecycle management.	Monitor AI services; evaluate models; manage embeddings/thresholds.

2.4 Operating Environment

Environment	Details
Client	Modern desktop browsers: Chrome, Edge, Firefox, Safari
Frontend	React, TypeScript, Vite, Tailwind CSS
Backend	Java 17/21, Spring Boot 3.x, Spring Data JPA (Hibernate), Python 3.11, FastAPI/Uvicorn, LangGraph
AI	Fine-tuned open-weight language model, external reasoning LLM provider, sentence-transformer embedding model
Data Storage	PostgreSQL, Chroma vector database
Deployment	Containerized Hybrid Monolith via Docker Compose; each service in an independent container communicating over internal REST/Docker networking

2.5 Design and Implementation Constraints

Category	Constraint
Architectural	Multi-agent orchestration via LangGraph; Hybrid Monolith of a Spring Boot API Gateway (REST APIs, auth, business logic, data access) and a Python FastAPI ML Sidecar (AI orchestration, LangGraph workflows, RAG);

	React/TypeScript frontend; PostgreSQL and ChromaDB persistence.
AI	External reasoning models shall never receive unredacted customer data; responses must be grounded in approved knowledge; AI outputs must be validated before presentation; unsupported responses must never be presented as verified.
Security	JWT authentication protects secured APIs; role-based authorization restricts privileged operations; secrets are never stored in source code; sensitive data is encrypted where applicable.
Deployment	Services are containerized, independently deployable, and environment-driven in configuration.

2.6 Assumptions and Dependencies

Assumptions:

- Users have reliable internet connectivity.
- Approved knowledge base documents are available before production deployment.
- External AI services remain available when configured.
- Human support personnel are available for escalated tickets.
- Organizational support policies remain externally maintained.

Dependencies: FastAPI, LangGraph, PostgreSQL, ChromaDB, Sentence Transformers, Docker, an external reasoning LLM provider, the fine-tuned classification model service, and approved knowledge base documents. Failure of any external dependency should degrade gracefully (escalation or a clear operational error) rather than produce unsupported responses.

3. Specific Requirements

This section specifies the functional and non-functional requirements for the Clario platform. Each requirement is uniquely identified to support traceability through development, testing, deployment, and maintenance. Requirements are grouped by functional area and presented with a unique ID, a concise “shall” statement, and an implementation priority.

3.1 Functionality

The Clario platform shall provide the functional capabilities listed below, organized by functional area. These requirements define the externally observable behavior of the system and form the baseline for implementation, verification, and validation.

3.1.1 User Authentication and Authorization

Requirement	Description	Acceptance Criteria	Priority
FR-001: User Authentication	The system shall authenticate registered users before allowing access to protected resources and services.	• Only valid users can authenticate. • Invalid credentials never produce tokens. • JWT tokens contain required claims.	High
FR-002: Authorization	The system shall authorize access to protected resources according to assigned user roles.	• Unauthorized users cannot access protected APIs. • Role restrictions are consistently enforced. • Authorization decisions are logged.	High
FR-003: Session Management	The system shall manage authenticated user sessions using JWT access and refresh tokens.	• Tokens expire according to configuration. • Refresh mechanism functions correctly. • Expired refresh tokens	Medium

		cannot be reused.	
--	--	-------------------	--

3.1.2 Ticket Management

Requirement	Description	Acceptance Criteria	Priority
FR-004: Ticket Submission	The system shall allow users to submit customer support tickets through the user interface or REST API.	• Tickets receive unique identifiers. • Validation rejects incomplete requests. • Submitted tickets appear in the ticket queue.	High
FR-005: Ticket Validation	The system shall validate incoming ticket data before initiating AI processing.	• Invalid requests are rejected. • Schema validation is deterministic. • Validation failures include descriptive error messages.	High
FR-006: Ticket Status Management	The system shall maintain and update the lifecycle status of every support ticket.	• Status transitions follow defined workflow rules. • Invalid transitions are rejected. • Complete status history is preserved.	High
FR-007: Ticket Retrieval	The system shall allow authorized users to retrieve ticket details and processing history.	• Authorized users can retrieve tickets. • Unauthorized users cannot access restricted tickets. • Retrieval operations are logged.	Medium

3.1.3 Ticket Classification

Requirement	Description	Acceptance Criteria	Priority
FR-008: Automatic Ticket Classification	The system shall automatically classify every newly submitted ticket using the fine-tuned classification model before agent orchestration begins.	• Every ticket is classified before routing. • Classification metadata is stored. • Model failures are recoverable.	High
FR-009: Priority Prediction	The system shall predict the operational priority of each support ticket.	• Every ticket receives a priority. • Priority is stored. • Low-confidence predictions are identifiable.	High
FR-010: Sentiment Detection	The system shall determine the overall customer sentiment expressed within the submitted ticket.	• Every processed ticket receives a sentiment label. • Sentiment prediction is stored with classification metadata.	Medium
FR-011: Confidence Score Generation	The system shall generate confidence scores for all classification predictions.	• Confidence score accompanies every prediction. • Confidence values remain within the configured range.	High
FR-012: Multi-Label Classification	The system shall support assigning multiple categories to a ticket when more than one business domain is identified.	• Multiple domains can be assigned. • Category ordering reflects confidence. • Personally Identifiable Information (PII) Detection and Redaction	Medium

3.1.4 PII Detection and Redaction

Requirement	Description	Acceptance Criteria	Priority
FR-013: PII Detection	The system shall identify personally identifiable information within tickets using regex and spaCy NER, storing mappings in a secure ShadowMap.	- Supported entity types are detected. - Detection metadata is retained.	High
FR-014: PII Redaction	The system shall redact detected PII before sending ticket content to external AI services.	- External AI never receives unredacted PII. - Original content remains securely stored.	Critical
FR-015: PII ResolvePass	The system shall restore original customer information when presenting responses to authorized users.	- Restoration is accurate. - Placeholder mappings remain protected. - Supervisor Agent Orchestration	High

3.1.5 Supervisor Agent Orchestration

Requirement	Description	Acceptance Criteria	Priority
FR-016: Workflow Initialization	The Supervisor Agent shall initialize the processing workflow for every validated ticket.	- Every valid ticket starts a workflow. - Workflow state is persisted.	High
FR-017: Dynamic Agent Routing	The Supervisor Agent shall dynamically determine which specialist AI agents should participate in ticket processing.	- Routing decisions use classification metadata. - Multiple specialists may execute when necessary.	High
FR-018: Workflow State Management	The Supervisor Agent shall maintain the complete execution state throughout the multi-agent workflow.	- Workflow state survives recoverable failures. - Complete execution history is available.	High
FR-019: Agent Coordination	The Supervisor Agent shall coordinate communication between participating AI agents.	- Agent interactions remain synchronized. - Communication failures are recoverable.	High
FR-020: Workflow Completion	The Supervisor Agent shall determine when the workflow has completed successfully or requires escalation.	- Every workflow ends in either: - Response generation, - Human escalation, or	High

3.1.6 Specialist AI Agents

Requirement	Description	Acceptance Criteria	Priority
FR-021: Specialist Agent Selection	The system shall invoke one or more specialist AI agents based on the ticket classification results.	- Agent selection is based on classification results. - Multiple agents may be selected when required. - Selection decisions are recorded.	High
FR-022: Specialist Reasoning	Each specialist AI agent shall analyze assigned tickets using domain-specific reasoning and approved organizational knowledge.	- Responses remain within the assigned business domain. - Reasoning utilizes retrieved knowledge. - Intermediate reasoning artifacts are stored.	High
FR-023: Multi-Agent Collaboration	The system shall support collaborative reasoning	Multiple agents may participate in one workflow.	High

	among multiple specialist agents when a ticket spans multiple domains.	• Final output combines all required domain expertise.	
FR-024: Agent Failure Recovery	The system shall recover from specialist agent failures without compromising workflow integrity.	• Agent failures do not corrupt workflow state. • Recovery attempts are logged. • Retrieval-Augmented Generation (RAG)	High

3.1.7 Retrieval-Augmented Generation (RAG)

Requirement	Description	Acceptance Criteria	Priority
FR-025: Knowledge Retrieval	The system shall retrieve relevant knowledge base documents before generating customer responses.	• Retrieval utilizes semantic similarity. • Document ranking is deterministic. • Retrieval metadata is stored.	High
FR-026: Context Construction	The system shall construct a contextual prompt using retrieved documents and ticket information.	• Context includes only relevant information. • Token budget is never exceeded. • Context sources remain traceable.	High
FR-027: Source Attribution	The system shall associate generated responses with the knowledge sources used during generation.	• Every grounded response references its supporting knowledge. • Source metadata is retained.	Medium
FR-028: Knowledge Version Tracking	The system shall maintain version information for every knowledge document used during response generation.	• Knowledge versions are preserved. • Historical workflows reference original document versions. • Response Validation	Medium

3.1.8 Response Validation

Requirement	Description	Acceptance Criteria	Priority
FR-029: Response Quality Validation	The system shall evaluate generated responses for completeness, relevance, and consistency.	• All generated responses are validated. • Validation results are stored.	High
FR-030: Hallucination Detection	The system shall identify unsupported statements within generated responses.	• Unsupported claims are detected. • Validation decisions are recorded.	Critical
FR-031: Reflection-Based Improvement	The system shall support bounded reflection to improve low-quality AI responses before escalation.	• Reflection attempts remain configurable. • Infinite reflection loops cannot occur.	Medium
FR-032: Final Response Approval	The system shall determine whether a response is suitable for customer delivery or requires human intervention.	• Only approved responses may be delivered. • Approval decisions are permanently recorded. • Human Review and Escalation	High

3.1.9 Human Review and Escalation

Requirement	Description	Acceptance Criteria	Priority
FR-033: Automatic Escalation	The system shall automatically escalate	• Escalated tickets appear in the human review queue.	High

	tickets to human personnel when AI processing cannot confidently resolve the request.	• Escalation reason is recorded. • Ticket ownership is updated.	
FR-034: AI Reasoning History Preservation	The system shall preserve the complete AI reasoning history associated with every escalated ticket.	• Processing history is complete. • Reasoning artifacts remain immutable. • Knowledge sources remain traceable.	High
FR-035: Human Response Editing	The system shall allow authorized personnel to review, modify, approve, or replace AI-generated responses before delivery.	• Human edits are preserved. • Original AI response remains available for auditing. • Approval history is recorded.	High
FR-036: Ticket Resolution	The system shall allow authorized personnel to resolve and close support tickets.	• Closed tickets become read-only. • Resolution metadata is retained. • Ticket lifecycle remains auditable.	High

3.1.10 Knowledge Base Management

Requirement	Description	Acceptance Criteria	Priority
FR-037: Knowledge Document Management	The system shall allow authorized users to create, update, archive, and delete knowledge base documents.	• Supported document formats are accepted. • Document metadata is validated. • Version history is maintained.	High
FR-038: Automatic Knowledge Indexing	The system shall automatically generate embeddings and update the vector database whenever approved documents change.	• New knowledge becomes searchable after indexing. • Failed indexing does not corrupt existing indexes.	High
FR-039: Knowledge Version Control	The system shall maintain historical versions of knowledge documents.	• Previous versions cannot be lost. • The active version is clearly identifiable.	Medium
FR-040: Knowledge Search	The system shall allow administrators to search and browse knowledge base documents.	• Metadata search is supported. • Search results are ordered by relevance. • User Management	Medium

3.1.11 User Management

Requirement	Description	Acceptance Criteria	Priority
FR-041: User Registration	The system shall allow administrators to create user accounts.	• Duplicate accounts are prevented. • User information is validated.	Medium
FR-042: Role Management	The system shall support role assignment and modification for authorized users.	• Role changes take effect immediately. • Permission updates are logged.	High
FR-043: Account Status Management	The system shall support activation, suspension, and deactivation of user accounts.	• Suspended users cannot authenticate. • Status changes are auditable. • System Administration	Medium

3.1.12 System Administration

Requirement	Description	Acceptance Criteria	Priority
FR-044: System Configuration Management	The system shall allow authorized administrators to manage configurable application settings.	• Configuration validation prevents invalid values. • Configuration changes are logged.	High
FR-045: Operational Dashboard	The system shall provide administrators with a dashboard displaying system status and AI workflow metrics.	• Dashboard updates with current operational information. • Critical failures are clearly highlighted.	Medium
FR-046: Audit Log Management	The system shall maintain a secure audit log of all administrative and security-sensitive operations.	• Audit records cannot be modified by ordinary users. • Administrative actions remain traceable. • Log integrity is preserved.	High

3.1.13 Logging and Monitoring

Requirement	Description	Acceptance Criteria	Priority
FR-047: System Event Logging	The system shall record all significant application events generated during normal operation.	• Application events are logged. • Log entries include timestamps and severity levels. • Logs remain searchable.	High
FR-048: AI Workflow Logging	The system shall record the execution history of every AI workflow.	• Workflow stages are traceable. • Agent execution order is preserved. • Processing duration is recorded.	High
FR-049: Security Event Logging	The system shall record authentication, authorization, and other security-sensitive events.	• Login attempts are recorded. • Authorization failures are logged. • Administrative actions remain auditable.	Critical
FR-050: Operational Monitoring	The system shall continuously monitor operational metrics for all major software services.	• Service metrics are updated continuously. • Threshold violations trigger alerts. • Monitoring history is retained.	High

3.1.14 Reporting and Analytics

Requirement	Description	Acceptance Criteria	Priority
FR-051: Ticket Analytics	The system shall generate analytical summaries of customer support tickets.	• Ticket statistics are accurate. • Reports include configurable date ranges.	Medium
FR-052: AI Performance Reporting	The system shall provide reports describing AI system performance.	• Reports include processing time. • Reports include escalation statistics. • Reports include validation outcomes.	Medium
FR-053: Export Reports	The system shall allow authorized users to export generated reports.	• Exported reports preserve report contents. • File generation is logged. • AI Model Management	Low

3.1.15 AI Model Management

Requirement	Description	Acceptance Criteria	Priority
FR-054: Model Registration	The system shall support registration of approved AI models for production use.	- Registered models contain complete metadata. - Duplicate registrations are prevented.	Medium
FR-055: Model Version Management	The system shall maintain version information for deployed AI models.	- Model versions remain traceable. - Previous versions remain recoverable.	Medium
FR-056: Model Health Monitoring	The system shall monitor the operational availability of AI models.	- Model failures are detected. - Health history is retained. - Operational alerts are generated.	High

3.1.16 Notification Services

Requirement	Description	Acceptance Criteria	Priority
FR-057: Workflow Notifications	The system shall notify users when significant ticket events occur.	- Notifications correspond to workflow events. - Delivery status is tracked.	Medium
FR-058: Administrative Alerts	The system shall notify administrators of critical operational conditions.	- Critical failures trigger alerts. - Alerts remain auditable. - Configuration Management	High

3.1.17 Configuration Management

Requirement	Description	Acceptance Criteria	Priority
FR-059: Runtime Configuration	The system shall allow authorized administrators to modify configurable operational settings.	- Invalid configuration values are rejected. - Configuration changes are logged.	Medium
FR-060: Configuration Backup and Recovery	The system shall support backup and restoration of configuration settings.	- Backup integrity is verified. - Restoration recreates previous configuration. - System Health Management	Low

3.1.18 System Health Management

Requirement	Description	Acceptance Criteria	Priority
FR-061: Health Check Service	The system shall expose health endpoints for all major software services.	- Health endpoints accurately reflect service status. - Response includes dependency health.	High
FR-062: Dependency Health Monitoring	The system shall continuously evaluate the availability of external dependencies.	- Dependency failures are detected promptly. - Degraded operation is supported where possible. - Health history is retained.	High

3.2 Usability

Usability requirements establish measurable criteria for interaction, interface consistency, accessibility, and user satisfaction.

ID	Requirement	Description	Key Requirements
UR-001	User Interface Consistency	Consistent UI (navigation, layout, behavior, design system) across all modules.	Navigation components shall maintain a consistent layout throughout the

			application. • Common user interface elements shall exhibit consistent behavior. • Colors, typography, icons, and spacing shall follow a unified design system.
UR-002	Ease of Learning	Minimize learning effort for first-time users through self-explanatory workflows and contextual guidance.	• Primary workflows shall be self-explanatory. • Frequently used functions shall be easily discoverable. • Contextual guidance shall be available where necessary.
UR-003	Efficient Task Completion	Enable users to complete common tasks with minimal interaction.	• Frequently performed operations shall require the fewest practical number of user interactions. • Search and filtering capabilities shall be available where large datasets are presented. • Frequently accessed information shall remain easily accessible.
UR-004	Responsive User Interface	Provide responsive user interactions during normal operation.	• User actions shall receive immediate visual feedback. • Loading indicators shall be displayed during long-running operations. • Progress indicators shall be shown for asynchronous processing tasks.
UR-005	Accessibility	Support accessibility principles to accommodate users with diverse abilities.	• User interface elements shall be keyboard accessible. • Text shall maintain sufficient contrast against background colors. • Images and icons shall include alternative text where appropriate.
UR-006	Error Prevention and Recovery	Assist users in preventing errors and recovering from incorrect operations.	• Required input fields shall be clearly identified. • Input validation shall occur before data submission whenever possible. • Confirmation dialogs shall be presented before destructive operations.
UR-007	User Feedback	Provide timely, informative feedback for all significant user actions.	• Successful operations shall display confirmation messages. • Failed operations shall present meaningful error messages. • System notifications shall clearly indicate the outcome of user actions.
UR-008	Dashboard Usability	Provide role-specific dashboards presenting information relevant to each	• Customers shall view submitted tickets and their current status.

		user category.	• Support agents shall view assigned and escalated tickets. • Knowledge Base Administrators shall access document management functions.
UR-009	Cross-Platform Compatibility	Provide a consistent user experience across supported desktop browsers.	• Interface layout shall remain consistent across supported browsers. • Core application functionality shall operate without browser-specific modifications. • Responsive layouts shall adapt appropriately to common desktop display resolutions.
UR-010	User Documentation and Help	Provide sufficient documentation and guidance to operate the platform effectively.	• User documentation shall describe major system features. • Administrative documentation shall explain system configuration and management procedures. • Context-sensitive help shall be available for complex workflows where appropriate.

3.3 Reliability

ID	Requirement	Description	Key Requirements
REL-001	System Availability	Maintain a high level of operational availability during normal business operation.	• Core application services shall remain continuously available except during scheduled maintenance. • Planned maintenance activities shall be communicated in advance whenever possible. • Service interruptions shall be minimized through controlled deployment procedures.
REL-002	Fault Tolerance	Continue operating safely when individual components experience recoverable failures.	• Failure of one internal component shall not cause complete system failure. • Independent services shall fail independently where applicable. • Failed requests shall return meaningful error responses rather than causing unexpected system termination.
REL-003	Workflow State Recovery	Preserve workflow execution state to support recovery from unexpected interruptions.	• Workflow state shall be persisted after each major processing stage. • Interrupted workflows shall be recoverable where technically feasible.

			• Recovery operations shall preserve workflow integrity.
REL-004	Data Integrity	Preserve the integrity and consistency of all stored application data.	• Database transactions shall maintain atomicity and consistency. • Invalid or partially processed data shall not be committed. • Concurrent operations shall not corrupt stored information.
REL-005	AI Workflow Reliability	Execute the AI processing pipeline reliably throughout the complete ticket lifecycle.	• AI workflow execution shall follow the defined orchestration sequence. • Individual agent failures shall be detected promptly. • Workflow failures shall result in controlled escalation rather than undefined behavior.
REL-006	External Dependency Reliability	Monitor and manage failures of external services and dependencies.	• Availability of databases, vector stores, AI providers, and supporting services shall be continuously monitored. • Temporary failures shall trigger retry mechanisms where appropriate. • Persistent failures shall initiate fallback procedures or human escalation.
REL-007	Error Recovery	Recover gracefully from recoverable operational errors.	• Recoverable errors shall not terminate application execution. • Failed operations shall return descriptive error information. • Recovery attempts shall follow predefined operational policies.
REL-008	Backup and Restore	Support reliable backup and restoration of critical application data and configuration.	• Configuration data shall be backed up periodically. • Application databases shall support backup and restoration procedures. • Restoration processes shall verify backup integrity before recovery.
REL-009	Logging Reliability	Ensure operational logs remain reliable and complete.	• Critical system events shall always be recorded. • Log entries shall include timestamps and severity levels. • Log storage failures shall not interrupt application execution.
REL-010	Service Health Monitoring	Continuously monitor the operational health of critical services.	• Critical system events shall always be recorded. • Log entries shall include timestamps and severity levels. • Log storage failures shall

			not interrupt application execution.
REL-011	Controlled System Shutdown	Perform controlled shutdown to prevent data corruption and preserve workflow integrity.	• Active transactions shall be completed or rolled back safely. • Workflow state shall be persisted before shutdown. • Database connections shall close gracefully.
REL-012	Continuous Operational Stability	Maintain stable operation during prolonged continuous execution.	• Long-running services shall manage memory and computational resources efficiently. • Background services shall operate without progressive degradation. • Resource utilization shall remain within acceptable operational limits.

3.4 Performance and Security

3.4.1 Performance Requirements

ID	Requirement	Description	Key Requirements
PER-001	API Response Time	Provide responsive API services during normal operating conditions.	• Authentication requests shall be processed efficiently. • Ticket management operations shall return responses within acceptable response times. • Administrative operations shall remain responsive during normal workloads.
PER-002	AI Workflow Processing Time	Complete AI-assisted ticket processing within an acceptable time for typical requests.	• Classification, orchestration, retrieval, generation, and validation shall execute as a coordinated workflow. • AI processing time shall be monitored for every ticket. • Processing bottlenecks shall be identifiable through operational metrics.
PER-003	Concurrent User Support	Support multiple simultaneous users without significant performance degradation.	• Authentication services shall support concurrent login requests. • Ticket submission and retrieval shall remain responsive during concurrent usage. • Administrative operations shall continue functioning during normal user activity.
PER-004	Database Performance	Efficiently process application transactions.	• Database queries shall be optimized using appropriate indexing. • Frequently accessed data shall be retrieved efficiently. • Long-running queries shall be monitored.
PER-005	Knowledge Retrieval	Retrieve relevant knowledge	• Semantic document

	Performance	efficiently within the RAG subsystem.	retrieval shall utilize vector similarity search. • Retrieved documents shall be ranked before response generation. • Retrieval latency shall be monitored.
PER-006	Resource Utilization	Utilize computing resources efficiently.	• CPU, memory, and storage utilization shall be monitored continuously. • Resource consumption shall remain within configured operational thresholds. • Resource exhaustion shall generate administrative alerts.
PER-007	Scalability	Support future growth without requiring major architectural modifications.	• Application services shall support horizontal scaling where appropriate. • AI processing components shall be independently deployable. • Database services shall support future performance optimization.
PER-008	Monitoring and Performance Metrics	Continuously collect operational performance metrics.	• Response times shall be measured. • AI workflow durations shall be recorded. • Resource utilization statistics shall be collected.

3.4.2 Security Requirements

ID	Requirement	Description	Key Requirements
SEC-001	User Authentication	Authenticate all users before granting access to protected resources.	• Authentication shall require valid user credentials. • JWT access and refresh tokens shall be used for session management. • Authentication failures shall be logged.
SEC-002	Authorization and Access Control	Enforce role-based access control (RBAC) for all protected operations.	• Permissions shall be assigned according to user roles. • Administrative functions shall be restricted to authorized personnel. • Access control decisions shall be enforced consistently.
SEC-003	Password Security	Securely manage user credentials.	• Passwords shall never be stored in plain text. • Passwords shall be hashed using an industry-standard cryptographic hashing algorithm. • Password verification shall occur only against stored password hashes.

SEC-004	Data Encryption	Protect sensitive information during storage and transmission.	• Communication between clients and servers shall use HTTPS. • Sensitive credentials shall be encrypted during transmission. • Configuration secrets shall not be stored within application source code.
SEC-005	PII Protection	Protect customer PII throughout the AI processing pipeline.	• PII shall be detected before external AI processing. • Sensitive information shall be redacted where required. • Authorized users shall receive restored information only after successful processing.
SEC-006	Audit Logging	Maintain secure audit records of security-sensitive operations.	• Authentication attempts shall be logged. • Authorization failures shall be recorded. • Administrative operations shall generate audit records.
SEC-007	Session Security	Securely manage authenticated user sessions.	• Access tokens shall expire after a configurable period. • Refresh tokens shall be validated before issuing new access tokens. • Expired or invalid tokens shall be rejected.
SEC-008	Input Validation	Validate all externally supplied input before processing.	• API requests shall undergo schema validation. • Invalid or malformed requests shall be rejected. • Input validation shall reduce the risk of common injection attacks.
SEC-009	Secure API Communication	Protect all API endpoints against unauthorized access.	• Protected endpoints shall require authentication. • Authorization shall be verified before request execution. • API requests shall be logged for security monitoring.
SEC-010	Secure AI Processing	Ensure AI-assisted processing complies with organizational security and privacy requirements.	• AI-generated responses shall be validated before delivery. • Retrieved knowledge shall originate only from approved organizational sources. • Unsupported or hallucinated AI responses shall not be presented as verified information.
SEC-011	Security Monitoring	Continuously monitor security-related events.	• Authentication failures shall be monitored. • Repeated unauthorized access attempts shall be logged.

			• Security anomalies shall generate administrative alerts.
SEC-012	Configuration Security	Protect application configuration and operational secrets.	• Configuration files containing sensitive information shall be protected. • API keys, database credentials, and cryptographic secrets shall be stored using secure configuration mechanisms. • Access to sensitive configuration shall be restricted to authorized administrators.

3.5 Supportability

ID	Requirement	Description	Key Requirements
SUP-001	Modular System Architecture	Enable components to be developed, tested, deployed, and maintained independently.	• Major system components shall have clearly defined responsibilities. • Components shall communicate through well-defined interfaces. • Changes to one component shall minimize the impact on other components.
SUP-002	Configuration Management	Support centralized and environment-specific configuration management.	• Operational parameters shall be configurable without modifying application source code. • Configuration values shall be stored externally using secure configuration mechanisms. • Separate configurations shall be supported for development, testing, and production environments.
SUP-003	Logging and Diagnostic Information	Provide comprehensive logging to support monitoring and fault diagnosis.	• Application logs shall record significant system events. • AI workflow execution shall be traceable through diagnostic logs. • Errors shall include sufficient contextual information for troubleshooting.
SUP-004	Health Monitoring	Provide mechanisms for monitoring the operational health of critical services.	• Health endpoints shall be available for major application services. • Internal dependency status shall be monitored continuously. • Service availability shall be visible through administrative interfaces.
SUP-005	Error Diagnosis	Facilitate efficient identification and diagnosis	• Error responses shall include unique identifiers

		of operational failures.	where appropriate. • Internal exceptions shall be recorded with sufficient diagnostic information. • Operational failures shall be categorized according to severity.
SUP-006	Deployment Support	Support repeatable and consistent deployment procedures.	• Software components shall support containerized deployment. • Deployment configuration shall remain environment independent. • Application startup and shutdown procedures shall be documented.
SUP-007	Maintainability	Structure software to facilitate future maintenance and enhancement.	• Source code shall follow established coding standards. • Components shall maintain clear separation of responsibilities. • Public interfaces shall remain well documented.
SUP-008	AI Model Support	Support maintenance and operational management of AI models.	• AI model versions shall be tracked. • Model metadata shall be maintained. • Model availability shall be monitored continuously.
SUP-009	Knowledge Base Maintenance	Support efficient maintenance of the organizational knowledge base.	• Knowledge documents shall support version control. • Document updates shall trigger controlled indexing procedures. • Historical document versions shall remain recoverable.
SUP-010	Operational Documentation	Provide sufficient documentation for deployment, operation, and troubleshooting.	• Administrator documentation shall describe system configuration procedures. • Deployment documentation shall describe installation and startup procedures. • API documentation shall remain synchronized with implemented interfaces.
SUP-011	Backup and Recovery Support	Support administrative backup and recovery procedures for critical data.	• Application configuration shall support backup and restoration. • Operational databases shall support backup procedures. • Recovery procedures shall preserve system consistency.
SUP-012	Extensibility	Support future functional expansion without significant architectural redesign.	• Additional specialist AI agents shall be integrated through the existing orchestration framework. • New knowledge sources

			shall be incorporated into the Retrieval-Augmented Generation pipeline using existing indexing mechanisms. • Additional APIs and user interface modules shall conform to the established architectural patterns.
--	--	--	--

3.6 Design Constraints

ID	Constraint	Description	Key Requirements
DC-001	Architectural Constraints	Conform to the approved modular, service-oriented architecture.	
DC-002	Technology Stack Constraints	Use the approved technology stack defined by the project architecture.	
DC-003	AI Processing Constraints	AI-assisted processing shall operate within defined architectural and operational boundaries.	
DC-004	Security Constraints	Comply with organizational security requirements throughout implementation.	
DC-005	Data Management Constraints	Manage persistent data according to approved database design principles.	
DC-006	Deployment Constraints	Support standardized deployment and operational management.	
DC-007	Interface Constraints	Software interfaces shall conform to established API and integration standards.	
DC-008	Coding and Development Standards	Follow established engineering practices (Git, PR review, CI, automated tests).	
DC-009	Operational Constraints	Operate within defined operational boundaries.	
DC-010	Scalability Constraints	Support future expansion without fundamental architectural redesign.	
DC-011	Compliance Constraints	Comply with applicable engineering standards, organizational policies, and project governance.	
DC-012	Third-Party Dependency Constraints	Manage external libraries, frameworks, and AI services in a controlled, maintainable manner.	
DC-013	Database Constraints	Adhere to the approved database architecture for consistency and integrity.	
DC-014	Vector Database Constraints	The RAG subsystem shall utilize the approved vector database implementation.	
DC-015	API Design Constraints	All backend services shall conform to approved API design standards.	
DC-016	AI Model Constraints	AI model integration shall	

		comply with the approved architecture.	
DC-017	RAG Constraints	The RAG pipeline shall follow the approved knowledge retrieval architecture.	
DC-018	User Interface Constraints	The UI shall comply with the approved frontend architecture and design system.	
DC-019	Desktop Application Constraints	The web application shall be built with React (Vite).	
DC-020	Authentication Constraints	Authentication mechanisms shall comply with the approved security architecture.	
DC-021	Logging Constraints	Logging implementation shall follow the approved observability architecture.	
DC-022	Configuration Constraints	Application configuration shall remain externally managed.	
DC-023	Development Constraints	Development shall follow Git-based version control, mandatory PR review, CI checks, and pre-deployment testing.	• The AI processing service shall support execution using standard CPU resources. • GPU acceleration may be utilized when compatible hardware is available. • AI processing components shall operate independently of the transactional backend.

3.7 On-line User Documentation and Help System Requirements

The system shall provide context-sensitive help tooltips within the React frontend, particularly within the Agent Review and Ticket Submission interfaces. An online user manual describing the ticket triage workflow and RAG knowledge ingestion process shall be hosted alongside the application deployment.

3.8 Purchased Components

There are no purchased software components required for the core execution of the Clario platform. The system relies on open-source frameworks and commercially available LLM APIs (e.g., Google Gemini) that are billed based on usage rather than purchased licensing.

3.9 Interfaces

The Clario platform shall provide well-defined interfaces supporting user interaction, software component integration, hardware compatibility, and communication between distributed services, ensuring reliability, security, and maintainability.

3.9.1 User Interfaces

Clario shall provide a modern, intuitive, role-based web GUI (React/Vite) that adapts to the authenticated user's role, provides immediate feedback, and maintains consistent layout, terminology, icons, and interaction patterns. Key screens include:

Interface	Purpose
Authentication	Login, credential validation, password recovery, session creation.
Dashboard	Role-specific overview: navigation, notifications, activity, AI workflow status.
Ticket Management	Ticket creation, search/filter, status/priority/category, comments, attachments, history.

AI Workflow	Workflow progress, classification results, retrieved knowledge, confidence score, validation status.
Human Review	Review/edit/approve/reject AI-generated responses; escalate; record review notes.
Knowledge Base Management	Document library, upload, metadata editing, versioning, indexing status.

3.9.2 Hardware Interfaces

- Network Interface: standard TCP/IP; Internet connectivity for cloud AI providers; localhost/Docker bridge networking between the Spring Boot app and Python ML Sidecar.
- Display Interface: standard desktop monitor resolutions with scalable, legible UI elements.
- Input Device Interface: standard keyboard, mouse/touchpad, and clipboard operations; no specialized hardware required.
- Containerized Deployment: Spring Boot application, Python FastAPI ML Sidecar, and PostgreSQL each run in Docker containers for environment parity.
- Hardware Compatibility: runs on standard desktop/server hardware; no proprietary devices, sensors, or dedicated AI accelerators required; GPU acceleration is optional.

3.9.3 Software Interfaces

Interface	Connected Components	Protocol / Format	Primary Functions
UI ↔ Backend	React frontend ↔ Spring Boot Backend	HTTPS/REST, JSON	Authentication, ticket management, dashboards, KB management, admin operations.
Backend ↔ ML Sidecar	Spring Boot ↔ Python FastAPI ML Sidecar	HTTP/REST (localhost or Docker network), JSON	Ticket classification, LangGraph orchestration, response generation, validation.
ML Sidecar ↔ External LLM	Python ML Sidecar ↔ external reasoning LLM provider	HTTPS/REST, JSON	Specialist reasoning and response generation using redacted ticket context.
Backend ↔ PostgreSQL	Spring Boot ↔ PostgreSQL	JDBC	Transactional storage of tickets, users, and logs.
ML Sidecar ↔ ChromaDB	Python ML Sidecar ↔ ChromaDB	Internal REST / client library	Embedding storage and semantic retrieval for RAG.

3.9.4 Communications Interfaces

- Authentication Communication: secure transmission of credentials; rejection of expired/invalid tokens; secure session renewal.
- Knowledge Base Communication: REST and local file-system access between document storage, ChromaDB, and the ML Sidecar for upload, indexing, embedding, and retrieval.
- Monitoring and Logging Communication: structured log events and health-check endpoints between services and the monitoring dashboard.
- Network Requirements: all communication over TCP/IP; secure Internet access for external services; trusted local networking internally; configurable timeouts and retries.
- Communication Security: HTTPS for all external communication; encrypted sensitive data in transit; tokens and API keys never transmitted or logged in plain text.
- Error Handling and Recovery: timeout detection, retries for transient failures, standardized error responses, and logging without exposing internal implementation details.

- Performance: minimal overhead between the Spring Boot application and the Python ML Sidecar; asynchronous processing for long-running AI operations.

3.10 Database Requirements

The system requires a relational database (PostgreSQL 16+) for transactional integrity of customer tickets, user accounts, and system logs using ACID properties. A vector database (ChromaDB) is required to store and index high-dimensional semantic embeddings (generated via all-MiniLM-L6-v2) supporting the RAG subsystem. Both databases must support containerized deployment via Docker volumes.

3.11 Licensing, Legal, Copyright, and Other Notices

The Clario platform utilizes open-source libraries governed by their respective permissive licenses (e.g., MIT, Apache 2.0). Any fine-tuned models derived from open-weight base models (such as Mistral or Llama) are subject to the acceptable use policies of their original creators. User data processed through external LLM providers must adhere to the data privacy and retention agreements specified by those providers.

3.12 Applicable Standards

- IEEE 29148:2018 – Systems and software engineering – Life cycle processes – Requirements engineering.
- REST Architectural Style – for internal service-to-service communication.
- OpenAPI Specification (OAS 3.0) – for API contract definitions between the Spring Boot gateway and the Python ML sidecar.
- JSON Web Token (JWT) Standard (RFC 7519) – for stateless authentication.

4. Supporting Information

4.1 Architecture References

Refer to the Clario Software Architecture Document (SAD) for detailed component diagrams, sequence flows, and exact deployment port mappings governing the Hybrid Monolithic structure.

4.2 Academic References

- Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” 2020.
- Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs,” arXiv:2305.14314, 2023.
- LangChain AI, “LangGraph: Multi-Agent State Workflows,” 2024.
- S. V. Jathanna, “SurrogateShield: Beyond Redaction for High-Utility, Privacy-Preserving LLM Interactions,” arXiv:2606.29567, 2026.
- L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” NeurIPS, 2023.
- J. Shen, “Evidence Graph Consistency in RAG,” arXiv:2606.06748, 2026.
- X. Xu et al., “A Survey on Knowledge Distillation of Large Language Models,” arXiv:2402.13116, 2024.
- E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” arXiv:2106.08685, 2021.
- Y. Li et al., “Mitigating Hallucination in Large Language Models,” arXiv:2510.24476, 2025.
- S. Han and W. Choi, “Multi-Agent Clinical Decision Support System for KTAS-Based Triage,” arXiv:2408.07531, 2024.

- Z. Yu et al., “Triangle: Empowering Incident Triage with Multi-LLM-Agents,” Microsoft Research, 2025.